



猜歌遊戲

網路程式設計 期末專題

資工二甲 1122928 呂天馨

專題簡介

想法來源

常常和朋友玩的猜歌遊戲。

使用 Python tkinter 製作 GUI 客戶端

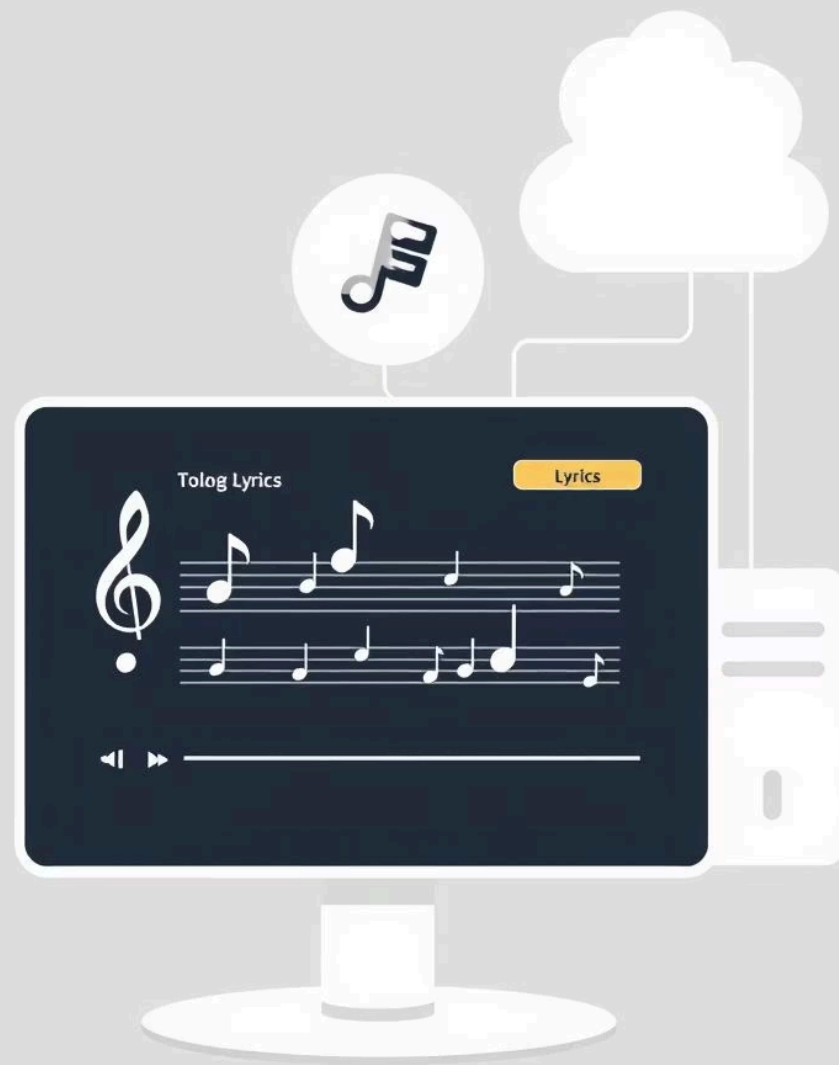
透過 Python 的 tkinter 庫，開發了一個直觀的圖形使用者介面，讓玩家能夠輕鬆地與遊戲互動。

使用 socket (TCP + UDP) 完成 client-server 通訊

遊戲的客戶端和伺服器之間採用 socket 進行通訊，結合 TCP 的可靠性和 UDP 的廣播特性，確保遊戲流程順暢。

支援多人同步競賽與記分板

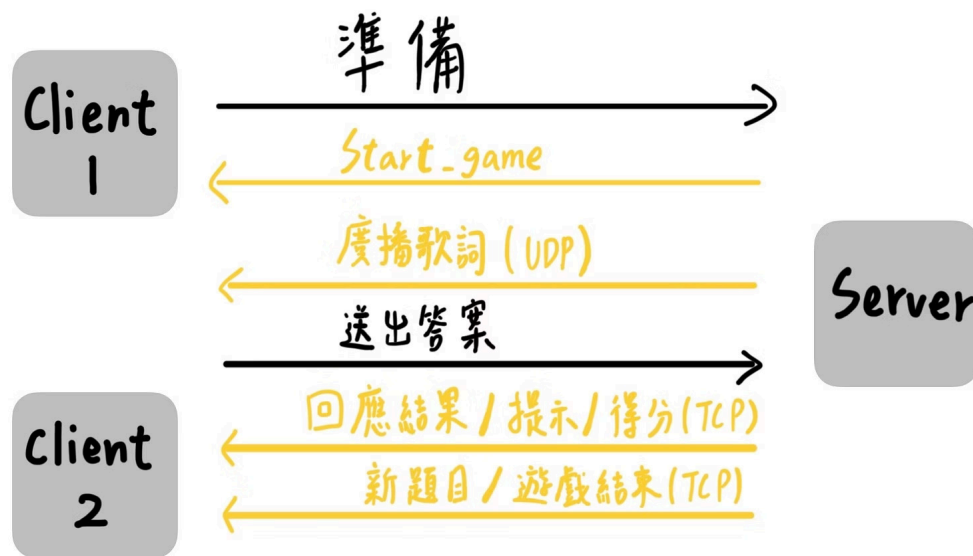
本遊戲支援多位玩家同時參與競賽，並設有即時更新的記分板，增加遊戲的互動性和競爭性。



遊戲規則

- **進入條件：**
 - 所有玩家連線並輸入名稱與頭像後，須點擊「我已準備」。
 - 全部玩家準備完成後，遊戲將於 3 秒後自動開始。
- **遊戲流程：**
 - 遊戲共 **5 回合**，每回合播放一首歌的歌詞片段。
 - 歌詞會每 **4 秒**廣播一句，由伺服器透過 **UDP Broadcast** 傳送。
- **玩家互動：**
 - 玩家可隨時猜「歌名」或「歌手」，答題欄下方有選擇按鈕。
 - 每項正確答案可得 **1 分** (若先答錯再答對，則不加分)。
- **回合結束條件：**
 - 若有人正確猜出歌名與歌手，該回合立即結束並進入下一題。
 - 若歌詞播畢仍無人完全答對，則自動公布正確答案，進入下一題。
- **遊戲結束：**
 - 所有題目結束後，顯示記分板與勝利者。
 - 玩家可點擊「返回大廳」重新進行下一輪遊戲。
- **特殊事件：**
 - 若任一玩家中途離線，遊戲 **強制中止**，並跳出提示。
 - 所有玩家將被送回大廳，重新準備遊戲。

系統架構圖





實際執行畫面展示



登入頁面

玩家輸入名稱後

可按下加入遊戲

進入遊戲大廳

猜歌小遊戲



歡迎 1 !

目前在線玩家：1

我已準備

離開遊戲

— 遊戲規則 —

1. 所有玩家都按下準備鍵，遊戲才會開始。
2. 系統將在畫面上方每 4 秒播放一句歌詞，共 5 回合。
3. 玩家可依據格式提問 (例如：歌名幾個字?)。
4. 每回合需分別猜中「歌名」與「歌手」，答對一個得 1 分。
5. 若先答錯再答對，則不加分。
6. 若均有人猜出，則提早進入下一題，否則直到歌曲結束才結束該回合。
7. 若有玩家在遊戲中退出，則遊戲中止，其餘玩家可返回遊戲大廳。

大廳



顯示頭像與玩家名稱

每個玩家在大廳中都會顯示其隨機的頭像和輸入的名稱。



準備、離開按鈕

提供按鈕讓玩家確認準備就緒，或是離開遊戲。

猜歌小遊戲



歡迎 1 !

目前在線玩家：1

我已準備

離開遊戲

— 遊戲規則 —

1. 所有玩家都按下準備鍵，遊戲才會開始。
2. 系統將在畫面上方每 4 秒播放一句歌詞，共 5 回合。
3. 玩家可依據格式提問 (例如：歌名幾個字?)。
4. 每回合需分別猜中「歌名」與「歌手」，答對一個得 1 分。
5. 若先答錯再答對，則不加分。
6. 若均有人猜出，則提早進入下一題，否則直到歌曲結束才結束該回合。
7. 若有玩家在遊戲中退出，則遊戲中止，其餘玩家可返回遊戲大廳。

大廳



遊戲規則

提供說明，讓玩家在進入遊戲前了解如何操作遊戲。

所有玩家按準備後，倒數三秒進入遊戲



猜歌介面

 猜歌遊戲

記分板

1 : 2 分 2 : 0 分

歌詞輸出

--- 第3 題 ---
別堆砌懷念讓劇情 變得狗血
深愛了多年又何必 毀了經典

☐ 猜歌名 ☒ 猜歌手

送出答案

提問 : 性別、國籍、歌手幾個字、歌名幾個字、是團體嗎 送出提問

 廣播：第 3 題開始囉，快來搶答！

猜歌介面

記分板	顯示各玩家目前的分數，一得分即更新
歌詞區	Server 端透過UDP每4秒廣播一句歌詞，共五輪
答題區	可在右側選擇回答歌名還是歌手，答對可獲得1分
提問區	可依提問格式，向系統提問： 性別、國籍、歌手幾個字、歌名幾個字、是團體嗎？ 若不符合格式，則會顯示提問格式有誤，請再試一次。
顯示輸出區	即時顯示當前題號，回答玩家的提問，以及哪位玩家成功答對了題目。若沒有玩家答對，則顯示正確答案。

⚠ 遊戲中有玩家離線，遊戲已中止

回到大廳

玩家離線



若中途有玩家離線，遊戲中止，其餘玩家可按按鈕回到大廳。

勝利者：1

1：6分

2：2分

回到大廳

成績結算畫面



顯示勝利者以及各玩家的分數，玩家可
回到大廳。

符合基本條件說明

TCP：建立 Client-Server 雙向通訊

TCP 用於客戶端與伺服器之間建立可靠的雙向連接，確保指令和重要數據的準確傳輸。

UDP：伺服器廣播歌詞

UDP 用於伺服器向所有連接的客戶端廣播歌詞，實現高效的即時資訊分發。

廣播方式：UDP broadcast

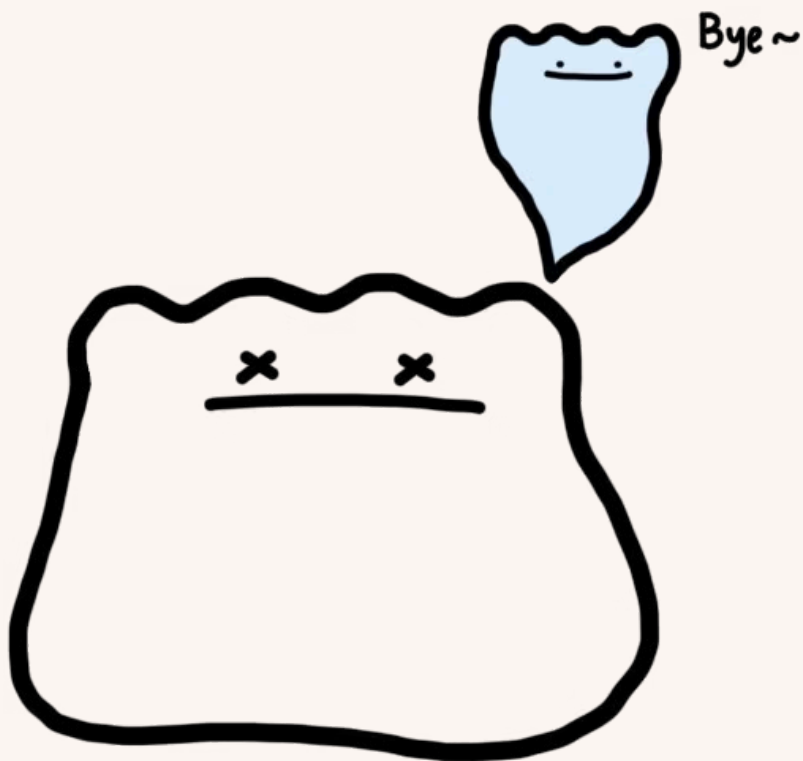
伺服器利用 UDP 廣播功能，將歌詞同時發送給所有在同一區域網路內的玩家，無需單獨發送。

多工：Server 使用 threading 處理多位玩家

伺服器採用多緒技術，為每位連接的玩家分配獨立的執行緒，實現多工處理，確保遊戲流暢。

GUI：Client 使用 tkinter

Client 端介面完全由 Python 的 tkinter 庫構建，提供圖形使用者介面，提升遊玩體驗。



困難與挑戰

多位玩家同步判斷與狀態管理困難

管理多位玩家的同步狀態，包括答題時間、分數計算和遊戲回合切換，是專案中一個複雜的挑戰

歌詞播放要一行一行同步，還要考慮提早答對就中止

歌詞是要每 4 秒播放一行，但如果大家都猜完了又要馬上停止，我一開始不知道怎麼同時控制時間與狀態

Tkinter 的 GUI 無法直接更新（會當掉）

一開始我在背景執行緒中直接更新畫面，結果程式會出錯或閃退

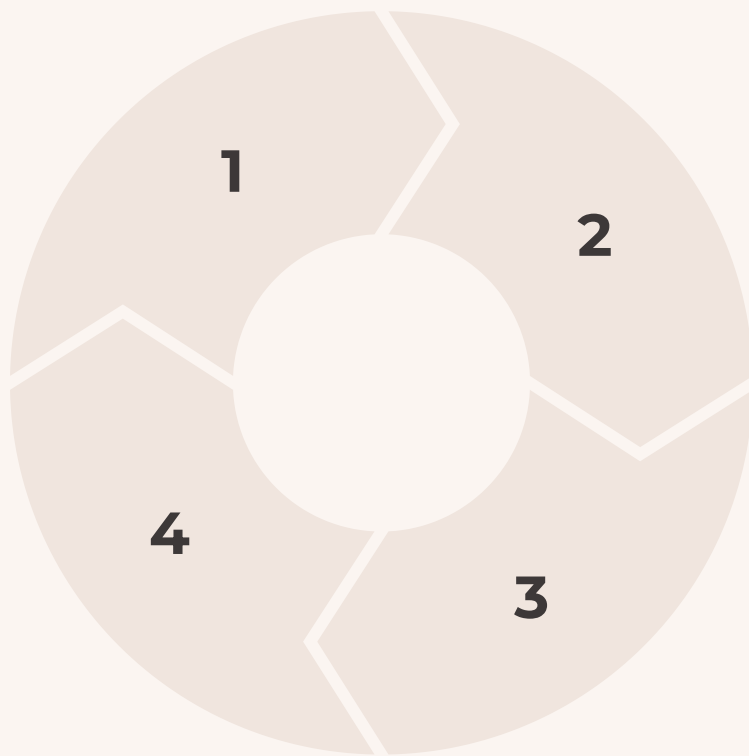
花最多時間的部分

設計 client 各狀態切換邏輯 (大廳 → 遊戲 → 結束)

客戶端在不同遊戲狀態（大廳、遊戲中、遊戲結束）之間的流暢切換邏輯設計，用了大量時間。

管理 server 端每題狀態與同步 結束邏輯

伺服器端需要精確管理每道題目的狀態，以及若有玩家中途離開的狀況，並確保所有玩家在遊戲結束時的同步邏輯。

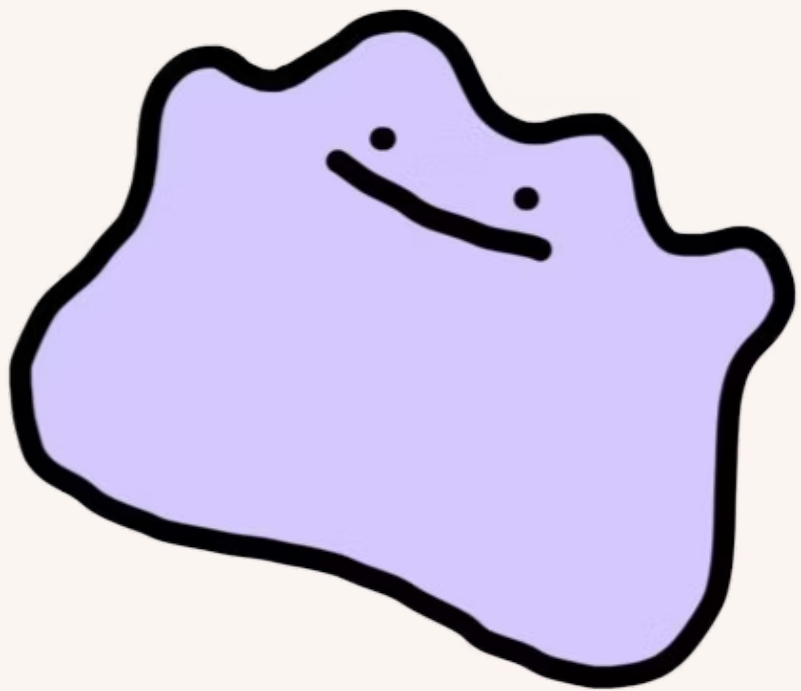


製作圖形介面：頭像、倒數動畫、記分版同步

圖形介面的視覺效果，包括頭像、遊戲開始倒數動畫和即時得分板的同步更新。

整理歌曲題庫

本來是請ChatGPT協助整理一百首中文流行歌曲，但後來發現資料有很多錯誤，所以最後還是自己整理。



自學的技術

1

設計倒數動畫畫面

畫面顯示「3、2、1 秒後開始遊戲」是透過 tkinter 的 `after()` 與 Label 文字更新完成的非同步動畫。

2

隨機分配玩家頭像功能

透過讀取 `avatars` 資料夾中的圖片（使用 Pillow 套件），每位玩家在加入時隨機分配一張頭像圖，並在 GUI 畫面中以 `ImageTk.PhotoImage` 方式呈現，提升遊戲的趣味性。

questions.json

```
1  [
2  {
3      "歌詞": "對這個世界如果你 有太多的抱怨\n跌倒了就不敢繼續往前走\n為什麼人要這麼的脆弱 墮落\n請你打開電視看看\n多少人為生命在努力勇敢的走下去\n我們是不是該知足\n珍惜一切 就算沒有擁有\n\n還記得你說家是唯一的城堡\n隨著稻香河流繼續奔跑\n微微笑 小時候的夢我知道\n不要哭讓螢火蟲帶著你逃跑\n鄉間的小路 永遠的依靠\n回家吧 回到最初的美好",
4      "答案": "稻香",
5      "歌手": "周杰倫",
6      "性別": "男",
7      "國籍": "台灣",
8      "類型": "個人"
9  },
10 {
11     "歌詞": "冷咖啡離開了杯墊\n我忍住的情緒在很後面\n拼命想挽回的從前\n在我臉上依舊清晰可見\n\n最美的不是下雨天\n是曾與你躲過雨的屋檐\n回憶的畫面 在盪著鞦韆 夢開始不甜\n\n妳說把愛漸漸 放下會走更遠\n又何必去改變 已錯過的時間",
12     "答案": "不能說的秘密",
13     "歌手": "周杰倫",
14     "性別": "男",
15     "國籍": "台灣",
16     "類型": "個人"
17  },
18 {
19     "歌詞": "我聽見雨滴落在青青草地\n我聽見遠方下課鐘聲響起\n\n可是我沒有聽見你的聲音\n認真呼喚我姓名\n\n愛上你的時候 還不懂感情\n離別了才覺得刻骨銘心\n\n為什麼沒有發現遇見了你\n你是生命最好的事情?\n\n也許當時笑著微笑和哭泣\n忙著追逐天空中的流星\n\n人理所當然的忘記\n是誰風裡 雨裡 一直默默守護在原地?",
20     "答案": "小幸運",
21     "歌手": "田馥甄",
22     "性別": "女",
23     "國籍": "台灣",
24     "類型": "個人"
25  },
26 {
27     "歌詞": "你是我最重要的決定\n我願意 每天在你身邊甦醒\n就連吵架也很過癮 不會冷冰\n\n因為真愛沒有輸贏 只有親密\n\n你是我最重要的決定\n我願意 打破對未知的恐懼\n就算流淚也能放晴 將心比心\n\n因為幸福沒有捷徑 只有經營",
28     "答案": "你是我最重要的決定",
29     "歌手": "范瑋琪",
```

自學的技術

3

音樂資料整理與 JSON 歌詞格式設計

學習如何整理大量的音樂資料，並設計一套標準的 JSON 格式來儲存和管理歌詞資訊。

4

PyInstaller 打包 GUI 與資源檔案

學習使用 PyInstaller 將 Python GUI 應用程式及其所有資源檔案打包成獨立的可執行檔。

程式碼簡要說明：Server 端

- 包含玩家管理、題庫載入、廣播歌詞、接收答案與分數統計
 - `broadcast(msg)`：將資料(倒數、答對、分數)廣播給所有 TCP 玩家
 - `udp_broadcast(text)`：透過 UDP 廣播歌詞句子
 - `handle_client(conn)`：以執行緒分別管理每個連線的玩家
 - `start_game()`：進入遊戲主流程（倒數、廣播、答題、統計）

Server

```
import socket
import threading
import json
import time
import random
import re

HOST, PORT = "0.0.0.0", 12346
UDP_PORT = 54321

clients = []
player_names = {}
player_avatars = {}
scores = {}
ready_players = set()

current_question = {}
guessed_correctly = {"title": False, "singer": False}
wrong_guesses = {}
current_round = 0
game_started = False
starter_conn = None
question_id_counter = 0

TOTAL_ROUNDS = 5
LYRIC_INTERVAL = 4
start_lock = threading.Lock()

with open("questions.json", "r", encoding="utf-8") as f:
    question_bank = json.load(f)

def broadcast(msg: dict):
    raw = (json.dumps(msg) + "\n").encode()
    dead = []
    for c in clients:
        try:
            c.sendall(raw)
        except:
            dead.append(c)
    for d in dead:
        remove_client(d)
```

載入 `questions.json` 題庫

def broadcast(msg: dict): 透過 TCP 傳送訊息給所有連線中的 client

- 遊戲倒數開始時傳送 {"type": "answer", "reply": "3 秒後開始遊戲..."}
- 每題答對時廣播 {"type": "result", "winner": "玩家1", "answer": "稻香"}
- 顯示排行榜與最終分數

Server

```
def udp_broadcast(text: str):
    try:
        udp_sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        udp_sock.setsockopt(socket.SOL_SOCKET, socket.SO_BROADCAST, 1)
        udp_sock.sendto(text.encode(), ("<broadcast>", UDP_PORT))
        udp_sock.close()
    except:
        pass

def send_to(conn, msg: dict):
    try:
        conn.sendall((json.dumps(msg) + "\n").encode())
    except:
        remove_client(conn)

def broadcast_online():
    player_list = [
        {"name": player_names[c], "avatar": player_avatars.get(c, 0)}
        for c in clients if c in player_names
    ]
    broadcast({"type": "online_list", "players": player_list})

def remove_client(conn):
    global game_started
    name = player_names.pop(conn, None)
    player_avatars.pop(conn, None)
    if conn in clients:
        clients.remove(conn)
    if name:
        scores.pop(name, None)
        wrong_guesses.pop(name, None)
        ready_players.discard(conn)
        print(f"[移除] {name} 離開，更新在線人數")

    if game_started:
        print("[警告] 有玩家離線，遊戲強制中止")
        broadcast({"type": "answer", "reply": "⚠ 有玩家離線，遊戲已被中止。"})
        broadcast({"type": "force_reset"})
        game_reset()

    broadcast_online()
    try:
        conn.close()
    except:
```

def udp_broadcast(text: str) : 使用 UDP 廣播將每句歌詞發送給所有 client

remove_client(conn) : 玩家中途離線，會移除其連線資訊，並根據當時是否在遊戲中決定是否強制中止整場遊戲，保護同步流程

Server

```
def handle_client(conn, addr):
    global game_started, starter_conn
    try:
        name = conn.recv(1024).decode().strip()
        if not name:
            return
        clients.append(conn)
        player_names[conn] = name
        print(f"[登入] {addr} -> {name}")
    except:
        remove_client(conn)
        return

    while True:
        try:
            data = conn.recv(4096).decode()
            if not data:
                break
            for line in data.strip().split("\n"):
                if not line.strip():
                    continue
                msg = json.loads(line)
                t = msg.get("type")

                if "avatar" in msg:
                    player_avatars[conn] = msg["avatar"]

                if t == "get_online":
                    send_to(conn, {
                        "type": "online_list",
                        "players": [
                            {"name": player_names.get(c), "avatar": player_avatars.get(c, 0)}
                            for c in clients if c in player_names
                        ]
                    })

                elif t == "ready":
                    ready_players.add(conn)
                    broadcast({"type": "player_ready", "player": player_names[conn]})
                    active_players = set(player_names.keys())
                    with start_lock:
                        if ready_players == active_players and not game_started:
                            starter_conn = conn
                            broadcast({"type": "answer", "reply": "3 秒後開始遊戲..."})
```

def handle_client(conn, addr) : 伺服器對每位 client 使用 threading

Thread 建立的主要監聽處理函式。包含接收暱稱、狀態同步（準備、答題、提問）、頭像登記與遊戲進入控制等。

Server

```
        guessed_correctly[cat] = True
        broadcast({
            "type": "result", "winner": name,
            "category": cat, "answer": ans, "scored": scored
        })
        broadcast({"type": "scoreboard", "scores": scores})
        if all(guessed_correctly.values()):
            threading.Timer(3, next_question).start()
    else:
        wrong_guesses[name][cat] = True

elif t == "question":
    q = msg["question"].lower()
    a = current_question
    reply = "提問格式有誤，請再試一次。"

    if "性別" in q:
        if a.get("類型") == "團體":
            reply = "這是一個團體，成員可能包含不同性別喔。"
        else:
            reply = f"歌手是 {a['性別']} 性。"
    elif "國籍" in q:
        reply = f"歌手是 {a['國籍']} 人。"
    elif "幾個字" in q and "歌名" in q:
        reply = f"歌名有 {len(a['答案'])} 個字。"
    elif "幾個字" in q and "歌手" in q:
        reply = f"歌手名字有 {len(a['歌手'])} 個字。"
    elif "團體" in q:
        reply = "是的，這是一個團體。" if a.get("類型") == "團體" else "不是，這位歌手是個人歌手。"
    elif a["歌手"] in q:
        reply = "是的。"

    broadcast({
        "type": "answer",
        "reply": f"{player_names[conn]} 提問：{q}\n👉 {reply}"
    })

elif t == "get_scoreboard":
    send_to(conn, {"type": "scoreboard", "scores": scores})

elif t == "exit_game":
    break
except:
    break
remove_client(conn)
print(f"[離線] {addr}")
```

guessed_correctly[cat] = True

broadcast({...})：若某位玩家猜中「歌名」或「歌手」，會將cat 設為 True，並廣播該玩家答對資訊與分數更新。若兩個類別皆猜中，會自動呼叫 next_question() 換下一題

各種問題格式判斷與回答產生：

- 性別 in q：依據 類型 回答是否為團體或個人。
- 國籍 in q：回答歌手國籍。
- 幾個字 in q：回答歌名字數。
- 幾個字 in q and 歌手 in q：回答歌手名字字數。
- 團體 in q：回答是否為團體。
- 歌手 in q：回答是否正確。

Server

```
def game_reset():
    global game_started, ready_players, current_round
    global current_question, guessed_correctly, wrong_guesses
    game_started = False
    ready_players.clear()
    current_round = 0
    current_question = {}
    guessed_correctly = {"title": False, "singer": False}
    wrong_guesses = {name: {"title": False, "singer": False} for name in player_names.values()}
    broadcast_online()

def start_game():
    global game_started, current_round, scores, wrong_guesses
    game_started = True
    current_round = 0
    scores = {name: 0 for name in player_names.values()}
    wrong_guesses = {name: {"title": False, "singer": False} for name in player_names.values()}
    broadcast({"type": "start_game_broadcast"})
    udp_broadcast("遊戲開始！準備好猜歌囉！")
    threading.Timer(1, next_question).start()

def next_question():
    global current_question, guessed_correctly, wrong_guesses, current_round, question_id_counter
    current_round += 1
    if current_round > TOTAL_ROUNDS:
        top = max(scores.values()) if scores else 0
        winners = [k for k, v in scores.items() if v == top]
        result = winners[0] if len(winners) == 1 else "平手"
        broadcast({"type": "game_over", "scores": scores, "winner": result})
        udp_broadcast(f"遊戲結束！{result} 獲勝！")
        game_reset()
        return

    current_question = random.choice(question_bank)
    question_id_counter += 1
    current_question["__id__"] = question_id_counter

    guessed_correctly = {"title": False, "singer": False}
    for k in scores:
        wrong_guesses[k] = {"title": False, "singer": False}

    broadcast({"type": "new_question", "round": current_round})
    udp_broadcast(f"第 {current_round} 題開始囉，快來搶答！")
    threading.Thread(target=play_lyrics, args=(current_question["__id__"],), daemon=True).start()
```

game_reset() :

- 初始化遊戲狀態（重設是否開始、分數、答對狀況等）
- 呼叫 `broadcast_online()` 通知更新狀態

start_game() :

- 重設回合、分數、錯誤紀錄
- 廣播「遊戲開始」

next_question() :

- 如果回合數大於五，則結算：
 - 找出最高分者或平手
 - 廣播「遊戲結束」與勝利者資訊
 - 呼叫 `game_reset()` 重設遊戲

Server

```
def play_lyrics(qid):
    lines = re.split(r"[•,\n]", current_question.get("歌詞", ""))
    lines = [l.strip() for l in lines if l.strip()]
    for line in lines:
        if not game_started or current_question.get("__id__") != qid:
            break
        if all(guessed_correctly.values()):
            break
        broadcast({"type": "lyrics_line", "line": line})
        time.sleep(LYRIC_INTERVAL)
    else:
        time.sleep(3)
        if game_started and current_question.get("__id__") == qid:
            title = current_question.get("答案", "未知歌名")
            singer = current_question.get("歌手", "未知歌手")
            broadcast({
                "type": "answer",
                "reply": f"🎉 本題結束，正確答案是：\n🎵 歌名：{title}\n🎤 歌手：{singer}"
            })
            threading.Timer(2, next_question).start()

# 主程式入口
print("[伺服器啟動] 等待玩家連線中...")
srv = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
srv.bind((HOST, PORT))
srv.listen()

while True:
    c, a = srv.accept()
    threading.Thread(target=handle_client, args=(c, a), daemon=True).start()
```

play_lyrics(qid):

- 播放歌詞邏輯：
 - 根據歌詞欄位逐句播放（每句間隔 LYRIC_INTERVAL 秒）。
 - 若提早全部猜中則中止播放。
 - 若播完仍未答對，則公布答案並 2 秒後自動進入下一題。

程式碼簡要說明：Client 端

- 包含畫面設計、玩家互動介面、TCP/UDP通信與答題邏輯
 - `setup_ui(self)`：設定介面框架（標題列、答題欄、準備與返回按鈕等）
 - `listen_server(self)`：用執行緒接收伺服器訊息並更新畫面
 - `udp_listener(self)`：使用 UDP 廣播接收歌詞
 - `show_countdown()`：所有人準備後的「3、2、1」倒數動畫
 - `handle_guess()`：處理玩家輸入的猜歌或猜歌手答案

Client

```
import socket
import threading
import tkinter as tk
from tkinter import ttk
from PIL import Image, ImageTk
import json
import os
import random
import traceback

HOST = "127.0.0.1"
PORT = 12346
UDP_PORT = 54321

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
AVATAR_DIR = os.path.join(BASE_DIR, "avatars")

RULES_TEXT = (
    "1. 所有玩家都按下準備鍵，遊戲才會開始。\\n"
    "2. 系統將在畫面上方每 4 秒播放一句歌詞，共 5 回合。\\n"
    "3. 玩家可依據格式提問（例如：歌名幾個字？）。\\n"
    "4. 每回合需分別猜中「歌名」與「歌手」，答對一個得 1 分。\\n"
    "5. 若先答錯再答對，則不加分。\\n"
    "6. 若均有人猜出，則提早進入下一題，否則直到歌曲結束才結束該回合。\\n"
    "7. 若有玩家在遊戲中退出，則遊戲中止，其餘玩家可返回遊戲大廳。"
)

class GuessSongClient:
    def __init__(self, master: tk.Tk):
        self.master = master
        self.master.title("猜歌遊戲")
        self.master.configure(bg="#f4f4f4")
        self.placeholder = "性別、國籍、歌手幾個字、歌名幾個字、是團體嗎"

        # TCP socket
        self.sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        try:
            self.sock.connect((HOST, PORT))
        except Exception:
            traceback.print_exc()
            self.master.destroy()
            return

        # UDP 廣播接收
        self.udp_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```

設定主視窗標題與背景色。

設定 placeholder 提示範例，幫助用戶輸入問題

建立 TCP socket 連接到伺服器

建立 UDP socket 接收歌詞廣播訊息（由伺服器 UDP 廣播）

標示遊戲規則的內容

Client

```
def listen_udp(self):
    while True:
        try:
            data, _ = self.udp_socket.recvfrom(1024)
            msg = data.decode()
            self.master.after(0, lambda: self.append_output(f"🔊 廣播: {msg}"))
        except:
            continue

def clear(self):
    for w in self.master.winfo_children():
        w.destroy()

def build_name_screen(self):
    self.master.geometry("300x200")
    self.master.minsize(300, 200)
    self.clear()
    ttk.Label(self.master, text="請輸入玩家名稱:").pack(pady=10)
    self.e_name = ttk.Entry(self.master, font=("微軟正黑體", 12))
    self.e_name.pack(pady=5)
    ttk.Button(self.master, text="加入遊戲", command=self.login).pack(pady=10)

def login(self):
    name = self.e_name.get().strip()
    if not name:
        return
    self.name = name
    self.sock.sendall((name + "\n").encode())
    self.build_lobby()

def build_lobby(self):
    self.master.geometry("800x500")
    self.master.minsize(700, 450)
    self.clear()
    ttk.Label(self.master, text="🎮 猜歌小遊戲 🎮", font=("微軟正黑體", 16, "bold")).pack(pady=10)
    if self.avatar_img:
        tk.Label(self.master, image=self.avatar_img, bg="#f4f4f4").pack(pady=5)
    ttk.Label(self.master, text=f"歡迎 {self.name}!", font=("微軟正黑體", 12)).pack(pady=5)
    self.online_label = ttk.Label(self.master, text="目前在線玩家: 等待回傳...", font=("微軟正黑體", 12))
    self.online_label.pack(pady=5)
    self.ready_btn = ttk.Button(self.master, text="我已準備", command=self.send_ready)
    self.ready_btn.pack(pady=5)
    ttk.Button(self.master, text="離開遊戲", command=self.exit_game).pack(pady=5)
    ttk.Label(self.master, text="— 遊戲規則 —", font=("微軟正黑體", 12)).pack(pady=(10, 0))
    tk.Message(self.master, text=RULES_TEXT, width=500, font=("微軟正黑體", 11), justify="left").pack(pady=5)
    self.send_json({"type": "get_online"})
```

listen_udp() :

- 不斷監聽伺服器從 UDP 發來的歌詞或提示
- 用 after() 將收到的廣播文字顯示到畫面中

clear() :

- 移除目前畫面所有元件，準備切換新畫面

build_name_screen() :

- 輸入玩家名稱畫面
- 顯示輸入框與「加入遊戲」按鈕

Client

```
def listen_udp(self):
    while True:
        try:
            data, _ = self.udp_socket.recvfrom(1024)
            msg = data.decode()
            self.master.after(0, lambda: self.append_output(f"🔊 廣播: {msg}"))
        except:
            continue

def clear(self):
    for w in self.master.winfo_children():
        w.destroy()

def build_name_screen(self):
    self.master.geometry("300x200")
    self.master.minsize(300, 200)
    self.clear()
    ttk.Label(self.master, text="請輸入玩家名稱:").pack(pady=10)
    self.e_name = ttk.Entry(self.master, font=("微軟正黑體", 12))
    self.e_name.pack(pady=5)
    ttk.Button(self.master, text="加入遊戲", command=self.login).pack(pady=10)

def login(self):
    name = self.e_name.get().strip()
    if not name:
        return
    self.name = name
    self.sock.sendall((name + "\n").encode())
    self.build_lobby()

def build_lobby(self):
    self.master.geometry("800x500")
    self.master.minsize(700, 450)
    self.clear()
    ttk.Label(self.master, text="🎮 猜歌小遊戲 🎮", font=("微軟正黑體", 16, "bold")).pack(pady=10)
    if self.avatar_img:
        tk.Label(self.master, image=self.avatar_img, bg="#f4f4f4").pack(pady=5)
    ttk.Label(self.master, text=f"歡迎 {self.name}!", font=("微軟正黑體", 12)).pack(pady=5)
    self.online_label = ttk.Label(self.master, text="目前在線玩家: 等待回傳...", font=("微軟正黑體", 12))
    self.online_label.pack(pady=5)
    self.ready_btn = ttk.Button(self.master, text="我已準備", command=self.send_ready)
    self.ready_btn.pack(pady=5)
    ttk.Button(self.master, text="離開遊戲", command=self.exit_game).pack(pady=5)
    ttk.Label(self.master, text="— 遊戲規則 —", font=("微軟正黑體", 12)).pack(pady=(10, 0))
    tk.Message(self.master, text=RULES_TEXT, width=500, font=("微軟正黑體", 11), justify="left").pack(pady=5)
    self.send_json({"type": "get_online"})
```

login(): 送出玩家名稱

- 若輸入名稱非空，將名稱透過 TCP 傳送給伺服器
- 登入後進入build_lobby()大廳畫面

build_lobby(): 遊戲大廳

- 顯示玩家名稱、頭像、在線玩家列表、規則說明
- 包含「我已準備」、「離開遊戲」按鈕

Client

```
def request_online(self):
    if not self.in_game:
        self.send_json({"type": "get_online"})
        self.master.after(2000, self.request_online)

def send_ready(self):
    self.send_json({"type": "ready"})
    self.ready_btn.config(text="✔ 已準備", state="disabled")

def build_game_ui(self):
    self.in_game = True
    self.clear()
    ttk.Separator(self.master, orient="horizontal").pack(fill=tk.X, pady=5)
    ttk.Label(self.master, text="🎯 記分板", font=("微軟正黑體", 14, "bold")).pack(pady=5)
    self.score_table = tk.Text(
        self.master, height=2, state="disabled", bd=0, highlightthickness=0,
        font=("微軟正黑體", 12), spacing1=4, spacing3=4
    )
    self.score_table.tag_configure("center", justify="center")
    self.score_table.pack(fill=tk.X, padx=10)
    self.send_json({"type": "get_scoreboard"})
    self.master.after(2000, self.request_scoreboard)

    lf = ttk.LabelFrame(self.master, text="歌詞輸出", padding=10)
    lf.pack(fill=tk.BOTH, expand=True, padx=10, pady=5)
    self.lyrics_display = tk.Text(
        lf, height=4, state="disabled", wrap="word",
        bd=0, highlightthickness=0, font=("微軟正黑體", 14), spacing1=6, spacing3=6
    )
    self.lyrics_display.tag_configure("center", justify="center")
    self.lyrics_display.pack(fill=tk.BOTH, expand=True)

    ttk.Separator(self.master, orient="horizontal").pack(fill=tk.X, pady=5)
    frm = ttk.Frame(self.master)
    frm.pack(fill=tk.X, padx=10, pady=5)
    self.guess_entry = ttk.Entry(frm, width=25, font=("微軟正黑體", 12))
    self.guess_entry.grid(row=0, column=0, padx=5)
    self.cat = tk.StringVar(value="title")
    ttk.Radiobutton(frm, text="猜歌名", variable=self.cat, value="title").grid(row=0, column=1)
    ttk.Radiobutton(frm, text="猜歌手", variable=self.cat, value="singer").grid(row=0, column=2)
    ttk.Button(frm, text="送出答案", command=self.send_guess).grid(row=0, column=3, padx=5)

    ttk.Label(frm, text="提問:", font=("微軟正黑體", 12)).grid(row=1, column=0, pady=5, sticky="w")
```

request_online()：要求在線玩家清單

- 若尚未進入遊戲，則每 2 秒透過 TCP 要求伺服器傳回最新玩家列表。

send_ready()：通知伺服器玩家已準備

- 傳送 {"type": "ready"} 給伺服器。
- 更新按鈕顯示為「✔ 已準備」並禁用。

build_game_ui()：建立遊戲進行中的畫面

- 分為兩個區塊：
 - 上半：顯示記分板與歌詞
 - 下半：玩家作答區

Client

```
def handle_message(self, msg: dict):
    t = msg.get("type")
    if t == "online_list":
        names = ", ".join(p["name"] for p in msg.get("players", []))
        if hasattr(self, "online_label"):
            self.online_label.config(text="目前在線玩家: " + names)
    elif t == "start_game_broadcast":
        self.build_game_ui()
    elif t == "new_question":
        rnd = msg.get("round", 1)
        if not self.in_game:
            self.build_game_ui()
        self.clear_question()
        self.lyric_queue.clear()
        self.lyrics_display.config(state="normal")
        self.lyrics_display.insert(tk.END, f"--- 第{rnd} 題 ---\n", "center")
        self.lyrics_display.config(state="disabled")
        self.master.after(3000, self.play_next_line)
    elif t == "lyrics_line":
        line = msg.get("line", "")
        if line:
            self.lyric_queue.append(line)
    elif t == "player_ready":
        self.append_output(f"✔ {msg.get('player')} 已準備")
    elif t == "result":
        tip = "✔" if msg.get("scored") else "❌"
        extra = "" if msg.get("scored") else "(已猜錯過, 0 分)"
        self.append_output(f"{tip} {msg.get('winner')} 猜對 {msg.get('category')}: {msg.get('answer')} {extra}")
    elif t == "answer":
        reply = msg.get("reply", "")
        if reply == "3 秒後開始遊戲...":
            self.show_countdown_screen(reply)
        else:
            self.append_output(f"❓ {reply}")
    elif t == "scoreboard":
        scores = msg.get("scores", {})
        self.score_table.config(state="normal")
        self.score_table.delete("1.0", tk.END)
        line = " ".join(f"{n}: {s} 分" for n, s in scores.items())
        self.score_table.insert(tk.END, line, ("center",))
        self.score_table.config(state="disabled")
        self.score_table.see(tk.END)
    elif t == "force_reset":
        self.in_game = False
```

handle_message(self, msg: dict) :

依照從 Server 收到的訊息類型
(msg["type"]) 來更新畫面與邏輯狀態。

Client

```
def append_lyrics(self, line: str):
    self.lyrics_display.config(state="normal")
    self.lyrics_display.insert(tk.END, line + "\n", "center")
    self.lyrics_display.see(tk.END)
    self.lyrics_display.config(state="disabled")

def append_output(self, text: str):
    self.output_text.config(state="normal")
    self.output_text.insert(tk.END, text + "\n")
    self.output_text.see(tk.END)
    self.output_text.config(state="disabled")

def show_countdown_screen(self, message):
    self.clear()
    self.master.configure(bg="white")

    self.countdown_label = ttk.Label(
        self.master,
        text=message,
        font=("微軟正黑體", 24, "bold"),
        foreground="#444"
    )
    self.countdown_label.place(relx=0.5, rely=0.5, anchor="center")

    self.countdown_value = 3
    self.update_countdown()

def update_countdown(self):
    if self.countdown_value <= 0:
        return # 等 server 發出 start_game_broadcast 再轉畫面
    text = f"{self.countdown_value} 秒後開始遊戲..."
    self.countdown_label.config(text=text)
    self.countdown_value -= 1
    self.master.after(1000, self.update_countdown)

def show_game_over(self, winner, scores: dict):
    self.in_game = False
    self.clear()
    ttk.Label(
        self.master,
        text=("平手" if winner == "平手" else f"勝利者: {winner}"),
        font=("微軟正黑體", 16, "bold")
    ).pack(pady=20)
    for n, s in scores.items():
        ttk.Label(self.master, text=f"{n}: {s} 分").pack()
    ttk.Button(self.master, text="回到大廳", command=self.build_lobby).pack(pady=20)
```

show_countdown_screen():

顯示倒數畫面（如「3 秒後開始遊戲...」）

show_game_over():

顯示誰是贏家及玩家最終得分

心得

這是我第一次實作出一款功能相對完善的遊戲，雖然介面簡單，但做起來真的好不容易。難以想像市面上的遊戲設計又會有多複雜。在一開始，我先大概做出雛形，後來慢慢加上很多平常在玩的時候會有的功能 (例如 : 顯示目前在線玩家、準備、提示功能)，遊戲漸漸成形，雖然一直在除錯，但也一直在學新東西，最後遊戲運行起來的時候，真的很感動！



THE END